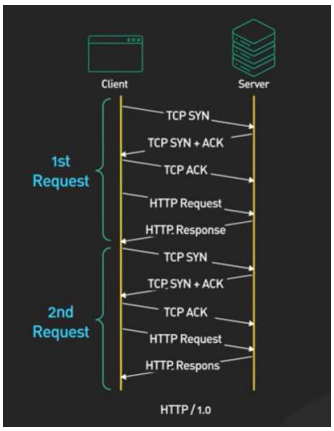
Networking / communications

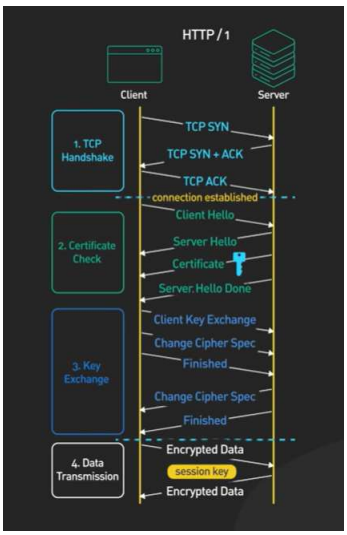
09 August 2026 12:01

Http -- hypertext transfer protocal

HTTP was initially for hypertext documents -- documents with links to other documents
But soon was used for images and videos -- and now people use for API / file transfer etC

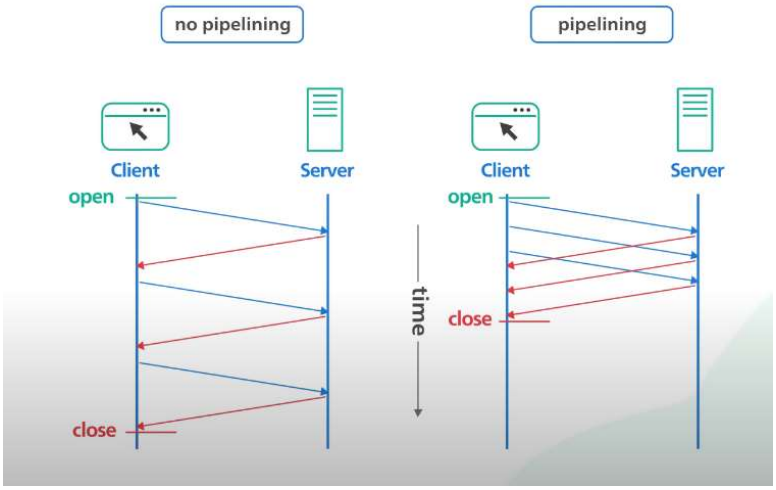
Http 1 -- Every request to the same server requires separate TCP connection -- meaning again sending SYN , SYN-ACK, and ACK packets again .





HTTP 1.1 - Introduced Connection: keep-alive in header so we don't need separate TCP connection for every request. It reduced latency. Server closes idle connection post come time automatically (Nginx - 75 seconds)

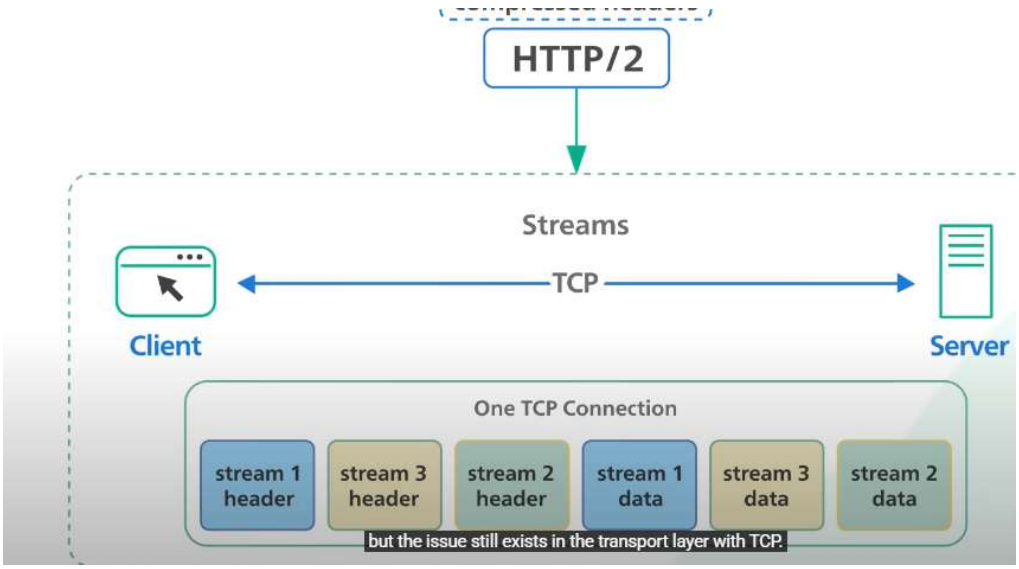
It also introduced pipelining -- we can send multiple request without waiting for response in single TCP. **This was removed afterwards as proxy could not handle this properly. They just use multiple TCP connection for performance improvement.**



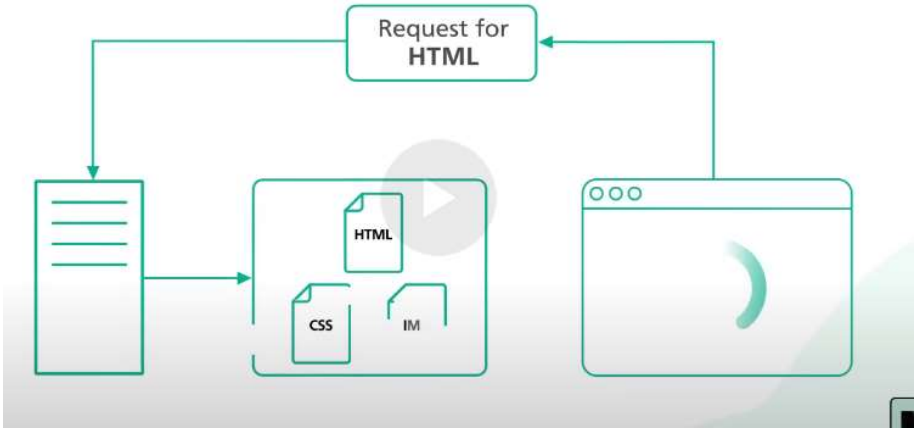
In HTTP/1.1, "Head-of-line blocking" (HOL blocking) occurs when a client's requests over a single TCP connection are held up because the response to the first request is delayed, even if subsequent requests could be processed faster. This happens because HTTP/1.1 requires responses to arrive in the same order as the requests, and if the first request takes a long time to process, it blocks all other requests on that connection

HTTP-2 : Introduces stream .

- Client can now send streams of data which are independent of each other in single TCP connection. It is spilted during transmission and put back on other end.
- The server will span up different virtual threads for different request - which are send via same TCP connection but different stream packet.
- It also supports push mechanism via which server can send new data when it is available without requiring client to poll. Spring Webflux along with netty implements this. Push mechanism also uses streams internally.



Server can push multiple response , unline http 1.1



HTTP-2 improvements over HTTP 1.1

- **Multiplexing:** Allows multiple requests and responses to be sent over a single TCP connection, eliminating the head-of-line blocking issue of HTTP/1.1. This internally uses streams.
- **Header Compression (HPACK):** Reduces the size of HTTP headers, leading to lower latency and bandwidth consumption. In Http 1.1 , it was plain text

Because HTTP/2 and HTTP/1.1 run on the same port (443), the client and server must agree on the protocol *before* sending any actual HTTP application data - this negotiation happens during the TLS handshake.

When we create REST client, we can specify

1. How many TCP connection can be made in parallel
2. For each TCP connection, how much time it can stay in queue before sending to server. (It reuses established TCP connections using standard HTTP Keep-Alive to send
3. How much time it can wait to get response from server

5 Max Connections, 2,000 Requests
HTTP 1.1 behavior

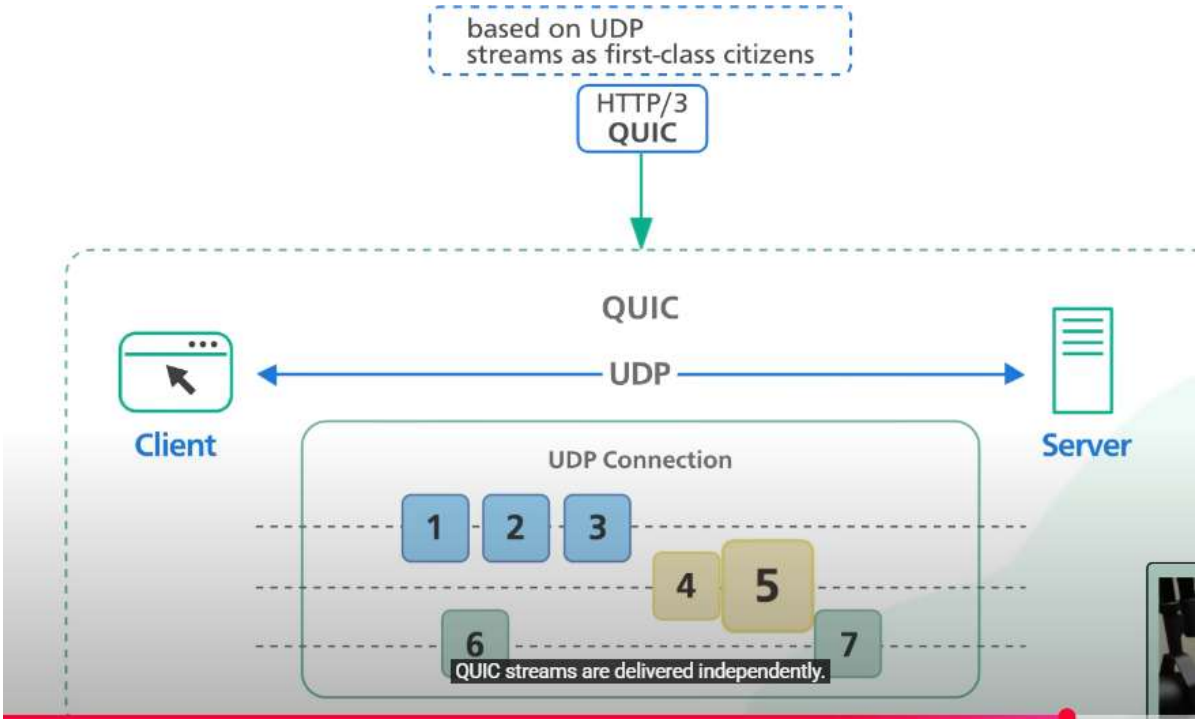
- **Requests 1–5:** Acquire the 5 available TCP connections from the pool, issue HTTP requests immediately, and wait for responses.
- **Requests 6–2000:** Sit in the client application's internal wait queue. The calling threads block (or suspend if using asynchronous processing) waiting for a connection to be checked back into the pool.
- **Timeout Risk:** If a queued request waits longer than the client's configured connectionRequestTimeout (the lease timeout), the thread unblocks and throws a ConnectionPoolTimeoutException or TimeoutException.

HTTP 2 behavior

SETTINGS_MAX_CONCURRENT_STREAMS Limit: Servers enforce a maximum number of concurrent active streams per TCP connection

- f the server limit is 100 streams per connection, 5 TCP connections can process **500 requests simultaneously** (5×100).
- Requests 1–500 are multiplexed across the 5 connections immediately.
- Requests 501–2000 enter the client's internal HTTP/2 stream queue.
- As active streams complete and close, queued requests immediately open new streams on the existing TCP connections.

HTTP-3 - (not much details) allows connections to persist even if the client's IP address changes (e.g., switching from Wi-Fi to cellular). It uses connection id to identify it. So even after ip change, we can use same connection to talk to server. This is useful in mobile devices etc, where we have internet problems. Also it can have multiple streams in same connection. - that's how it solves head of line blocking.
QUIC is build on top of UDP
HTTP-3 is faster in setting up connection -during handshake etc



CORS : Cross-Origin Resource Sharing error

- For every domain , cookies and session tokens are stored in browser against domain.
- Website cannot read cookies belonging to other domains. JavaScript code running on your website (attacker.com) only has access to cookies that were explicitly set for attacker.com. It cannot view, read, or export cookies stored for yourbank.com, google.com, or any other domain.

- The browser attaches your logged-in session cookie for yourbank.com to that request automatically -- earlier now its not the case.
- Whenever website (attacker.com) makes call to another domain yourbank.com , browser will attach session cookies and origin request header (origin : attacker.com) . If server has Access-Control-Allow-Origin : attacker.com in response header, then browser treat call as valid and allows attacker.com to read result.
- 'access-control-allow-origin' will have only 1 domain same as 'origin' or regex , it cannot have list of domain
- For non-simple requests (like PUT, DELETE, or custom JSON headers), the browser sends an HTTP OPTIONS "preflight" check first. If the target server does not authorize the origin, the actual request is never sent, protecting state-changing actions. Request / response body both are of size 0.
- If scheme(http/hhttps) , port (433, 8080) and domain is same, then it is considered from same endpoint and no CORS error. Additionally, example.com and sub.example.com are different domain.

Why server don't pro-actively check 'origin' request header and send 403 error

1. Client can be anything (curl, mobile devive etc), browser is just one of them.
2. Backward compability -- old server may not be checking origin or sending 'Access-Control-Allow-Origin' response. Hence no security risk if browser is proper.
3. Server with new strict code can do this strict enforcement.

Websocket

- WebSocket is a **stateful** protocol that runs directly on top of **TCP** (or TLS/TCP for encrypted wss://), not on top of HTTP/2.
- It is duplex, two- way , meaning anyone can send data
- Used by real time web application -> stock market, chat apps, gaming
- First handshake is done by http (in request header it mentions Upgrade: websocket) , once handshake is completed, it converts to duplex websocket.
- Anyone can close the connection afterwards
- WebSockets are hard to scale because
    - they are stateful and persistent.
    - Unlike normal HTTP requests that open and close quickly, a WebSocket keeps an open TCP connection for every user.
    - This forces servers to track memory, CPU, and network states for millions of concurrent users at the same time.
    - Even when no data moves, servers must send regular ping/pong heartbeat signals to make sure lines are still alive, adding background work
- If we don't use websocket, we have to use polling.
- Short polling -- asking server and server says no in continuous interval (say gap of 5 sec)
- Long polling- ask server,the server holds connection for few minutes and then says no or sends all data at once. In continous interval

SSE - server send events :

-
- Instead of application/json , it uses text/event-stream as content type

- Unlike traditional HTTP, where there is request/response, here its Unidirectional Push (Server streams data)
- In standard HTTP, the client sends a request (e.g., GET /data), the server returns a complete response, and the connection finishes or goes idle. With SSE, the client sends an initial request, but the server responds with a header set to Content-Type: text/event-stream. Instead of closing the connection, the server keeps it open using HTTP chunked transfer encoding, streaming new data blocks down to the client as events occur.
- AI models (such as OpenAI, Claude, and local LLMs) stream text token-by-token rather than waiting for full responses. SSE is the industry-standard transport protocol for LLM response streaming due to its low overhead and native text streaming support.
- For one-way server updates (e.g., live dashboards, sports scores, price tickers), WebSockets add unnecessary complexity.
- SSE has to be base-64, web-socket can work with binary data, hence payload size increases by 33%
- MCP also uses MCP
    - Initial SSE Setup (Server $\rightarrow$ Client)
        - The client initiates a single GET request to establish the SSE stream.
        - The server keeps this connection open indefinitely.
        - Upon connection, the server immediately emits an endpoint event over the SSE stream, providing a target URI (often including a unique session ID, e.g., /messages?sessionId=xyz).

      Incoming Requests (Client $\rightarrow$ Server)
        - it opens a short-lived **HTTP POST** request aimed at the assigned session URI.
        - Multiple POST call for multiple Request

      Response of POST

      The server processes the POST request and sends the JSON-RPC response back over the **existing persistent SSE stream** rather than in the HTTP POST response body
      Any asynchronous server notifications or logs also travel through this same unified SSE stream.

JSON -RPC

- **JSON-RPC** (JSON Remote Procedure Call) is a lightweight, stateless protocol that allows a client to execute a function or method on a remote server and receive the result back
- Unlike REST, which manipulates resources, it is action-oriented

  Input (send via REST POST)
  { "jsonrpc": "2.0", "method": "getUserProfile", "params": {"userId": 42}, "id": 1 }
- jsonrpc: Must be exactly "2.0".
- id: An identifier (string, integer, or null) matching requests to responses.

  Output (via SSE)
  { "jsonrpc": "2.0", "result": { "userId": 42, "name": "Alice", "role": "Admin" }, "id": 1 }

  Or

  {"jsonrpc": "2.0", "method": "add", "params": [2, 3], "id": 1}, {"jsonrpc": "2.0", "method": "subtract", "params": [10, 4], "id": 2}]

  It can be batch response (response of multiple mcp call) , notifications (basically intermediate thinking), error response etc.

•